

REPORT 2° HOMEWORK

Introduction

In this homework, it was required to create a script that uses an **OpenSSL** API to implement the encryption/decryption algorithms (in particular **AES**, **Camellia**, **SM4**) on different sizes of file `.txt` (`16B`, `20KB`, more than `2MB`) in **CBC** mode with a fixed **128 bit size symmetric key**. In particular, *we want to compare the performance for each algorithm*. Note that, because of the 128 bits length of the key, it was used **AES-128**.

Explanation

First, it was created a script `generateFiles.c` that creates 3 files in the `../files/` directory:

- `firstFile.txt` which is a `16B` of file
- `secondFile.txt` which is a `20KB` of file
- `thirdFile.txt` which is more than `2MB` of file

Once generated those files, it is possible to encrypt and to decrypt those and test the performance after compiling `test.c` file. It tests the performance of encryption and decryption for each algorithm to each file. In particular, it was chosen to randomise the **symmetric key** (of **128 bit**) and also the **Inizialization Vector (IV)** of **16 bit**.

For implementing an OpenSSL API, it was chosen the `libcrypto` library, which provides 3 different functions for each encryption (`EVP_EncryptInit_ex`, `EVP_EncryptUpdate`, `EVP_EncryptFinal_ex`) and decryption (`VP_DecryptInit_ex`, `EVP_DecryptUpdate`, `EVP_DecryptFinal_ex`) which are specular:

- `EVP_EncryptInit_ex` / `VP_DecryptInit_ex`, initialise the context
- `EVP_EncryptUpdate` / `EVP_DecryptUpdate`, starts encrypting/decrypting in blocks of `EVP_CIPHER_block_size` which depends by the chosen encryption/decryption algorithm, and the `IV` size has to be equals to it. In this case, because of **AES/Camellia/SD4**, `EVP_CIPHER_block_size` is equals to 16 bit. Note that because of this fixed size, may happen that some plaintext/ciphertext's bits won't be encrypted in this function and so it will

be later in the last stage with some padding (in this case, it was chosen to leave enabled the padding as default, which consider the PKCS#7 padding style).

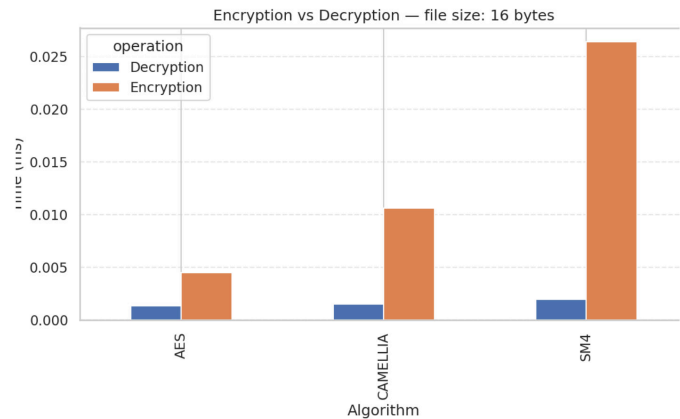
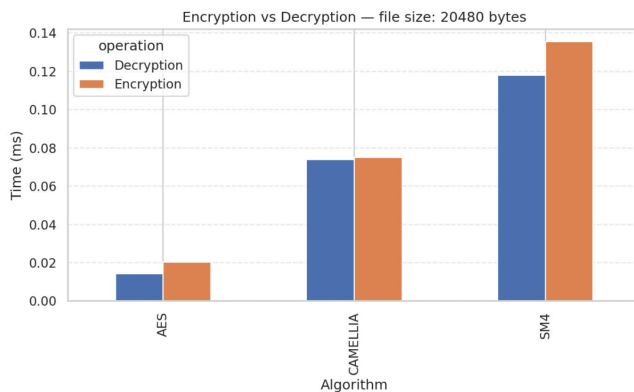
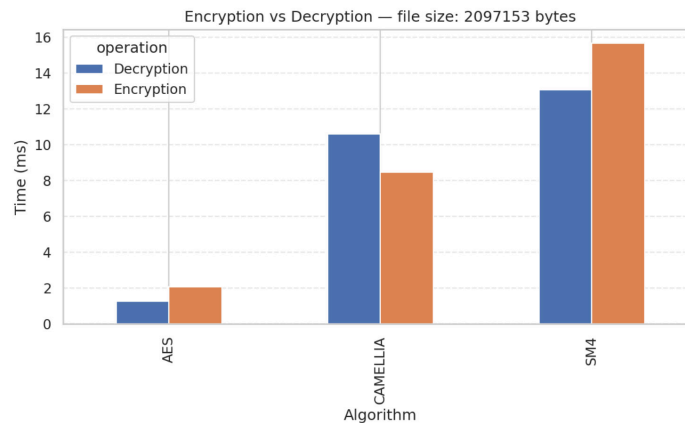
- `EVP_EncryptFinal_ex` / `EVP_DecryptFinal_ex` , finishes to encrypt/decrypt the last bits padded.

To measure the performance, simply has been measured the current real time (wall clock) in milliseconds before and after the execution of the algorithm, computed the difference between those values and returned it.

In the end of the scripts, the user will receive an output like that:

```
-----STARTING TESTING-----  
TYPE - ALGORITHM - FILE_SIZE - RESULT(ms)  
Encryption - AES - 16 - 0.004488  
Decryption - AES - 16 - 0.001363  
Encryption - AES - 20480 - 0.020338  
Decryption - AES - 20480 - 0.014277  
Encryption - AES - 2097153 - 2.077194  
Decryption - AES - 2097153 - 1.282958  
Encryption - CAMELLIA - 16 - 0.010640  
Decryption - CAMELLIA - 16 - 0.001513  
Encryption - CAMELLIA - 20480 - 0.075011  
Decryption - CAMELLIA - 20480 - 0.073719  
Encryption - CAMELLIA - 2097153 - 8.465779  
Decryption - CAMELLIA - 2097153 - 10.586073  
Encryption - SM4 - 16 - 0.026389  
Decryption - SM4 - 16 - 0.002004  
Encryption - SM4 - 20480 - 0.135435  
Decryption - SM4 - 20480 - 0.117872  
Encryption - SM4 - 2097153 - 15.652388  
Decryption - SM4 - 2097153 - 13.063711  
-----
```

Graphic Interpretation (the lower is better)



Let's consider each algorithm for either encryption and decryption:

- **AES-128:** It is by far the best of three. The larger the file size to be encrypted, the more evident the performance difference becomes. Considering that AES is considered very secure, does not have any disadvantages.
 - **CAMELLIA:** it is the second best algorithm. It is considered very secure, but because is less optimized than AES, results slower.
 - **SM4:** Is the slowest in each comparison. Considering also that is not as secure as the other competitor, is not recommended outside the China.
-

Code

generateFiles.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <sys/stat.h>

#define SIZE_1 (16)           //16B
#define SIZE_2 (20*1024)     //20KB
#define SIZE_3 ((2*1024*1024)+1) // more than 2MB

void fill_document(FILE* fd, int size){
    for (int i=0; i<size;i++) putc('A',fd); //i could use fwrite with a
    string of 3MB size to be more efficient
}

int ensure_dir_exists(const char *path) {
    struct stat st;
    if (stat(path, &st) == 0) {
        if (S_ISDIR(st.st_mode)) return 0;
        errno = ENOTDIR;
        return -1;
    }
    if (mkdir(path, 0755) == 0 || errno == EEXIST) return 0;
    return -1;
}

int main(){
    //checks if ../file directory exists
    if (ensure_dir_exists("../files") != 0) {
        fprintf(stderr, "ERROR: cannot ensure ../files directory: %s\n",
        strerror(errno));
        return 1;
    }

    //16B
    FILE* fd = fopen("../files/firstFile.txt", "w");
    if (!fd){
        fprintf(stderr, "ERROR: failed to open the file in
        ../files/firstFile.txt");
        return 1;
    }
}
```

```

    }
    fill_document(fd, SIZE_1);
    fclose(fd);

    //20KB
    fd = fopen("../files/secondFile.txt", "w");
    if (!fd){
        fprintf(stderr, "ERROR: failed to open the file in
../files/secondFile.txt");
        return 1;
    }
    fill_document(fd, SIZE_2);
    fclose(fd);

    //2MB
    fd = fopen("../files/thirdFile.txt", "w");
    if (!fd){
        fprintf(stderr, "ERROR: failed to open the file in
../files/thirdFile.txt");
        return 1;
    }
    fill_document(fd, SIZE_3);
    fclose(fd);

    //finishing
    return 0;
}

```

test.c

```

#define _POSIX_C_SOURCE 199309L //this is added for visual code purposes
#include <stdio.h>
#include <stdlib.h>
#include <time.h>           //we need this to measure time
#include <openssl/evp.h>    // ... for encrypt/decrypt
#include <openssl/rand.h>   // ... for randomize KEY and IV
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <string.h>

#define KEY_BITS 128      //128bit for the key
#define KEY_LENGTH (KEY_BITS/8) //in bytes
#define IV_LENGTH 16      //16 bit, because we chose AES/MD4/Camellia works
with 16bit blocks

```

```

#define FILE_SIZE_1 (16) //16B
#define FILE_PATH_1 "../files/firstFile.txt"

#define FILE_SIZE_2 (20*1024) //20KB
#define FILE_PATH_2 "../files/secondFile.txt"

#define FILE_SIZE_3 ((2*1024*1024)+1) //more than 2MB
#define FILE_PATH_3 "../files/thirdFile.txt"

#define AES "aes-algorithm"
#define CAMELLIA "camelia-algorithm"
#define SM4 "sm4-algorithm"

typedef struct cipher_data{
    unsigned char* plaintext;
    unsigned char* ciphertext;
    int plaintext_len;
    int ciphertext_len;
    unsigned char* key;
    unsigned char* iv;
} cipher_data;

void myfree(cipher_data* cd, int fd){
    close(fd);
    if (cd->plaintext) { free(cd->plaintext); cd->plaintext = NULL; }
    if (cd->ciphertext) { free(cd->ciphertext); cd->ciphertext = NULL; }
}

double get_real_time_msec() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec*1E03 + ts.tv_nsec*1E-06;
}

double testing_encryption(const char* algorithm, cipher_data* cd){
    double start = get_real_time_msec();
    unsigned char* iv = cd->iv;
    unsigned char* key = cd->key;

    //initializing contex
    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    if (!ctx) {
        fprintf(stderr, "ERROR: EVP_CIPHER_CTX_new failed\n");
        return -1;
    }
}

```

```

const EVP_CIPHER *cipher = NULL;

if (strcmp(algorithm, AES) == 0) {
    cipher = EVP_aes_128_cbc();          /* uses 16-byte key */
} else if (strcmp(algorithm, CAMELLIA) == 0) {
    cipher = EVP_camellia_128_cbc();
} else if (strcmp(algorithm, SM4) == 0) {
    cipher = EVP_sm4_cbc();
} else {
    fprintf(stderr, "ERROR: unknown algorithm: %s\n", algorithm);
    EVP_CIPHER_CTX_free(ctx);
    return -1;
}

if (1 != EVP_EncryptInit_ex(ctx, cipher, NULL, key, iv)) {
    fprintf(stderr, "ERROR: EVP_EncryptInit_ex failed\n");
    EVP_CIPHER_CTX_free(ctx);
    return -1;
}

//encrypting
int block_size = EVP_CIPHER_block_size(cipher);
unsigned char *out = malloc((size_t)cd->plaintext_len + block_size);
if (!out) {
    perror("Something went wrong with the malloc inside EVP_Encrypt");
    EVP_CIPHER_CTX_free(ctx);
    return -1;
}

int out_len = 0, tmplen = 0;
if (cd->plaintext_len > 0) {
    if (1 != EVP_EncryptUpdate(ctx, out, &out_len, (unsigned char*)cd-
>plaintext, cd->plaintext_len)) {
        fprintf(stderr, "ERROR: EVP_EncryptUpdate failed\n");
        free(out);
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }
}

//Finalizing padding block
if (1 != EVP_EncryptFinal_ex(ctx, out + out_len, &tmplen)) {
    fprintf(stderr, "ERROR: EVP_EncryptFinal_ex failed\n");
    free(out);
    EVP_CIPHER_CTX_free(ctx);
    return -1;
}

```

```

    }
    out_len += tmplen;

    cd->ciphertext=out;
    cd->ciphertext_len=out_len;

    //freeing contex
    EVP_CIPHER_CTX_free(ctx);

    double elapsed = get_real_time_msec() - start;
    return elapsed;
}

double testing_decryption(const char* algorithm, cipher_data* cd){
    double start = get_real_time_msec();
    unsigned char* iv = cd->iv;
    unsigned char* key = cd->key;

    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    if (!ctx) {
        fprintf(stderr, "ERROR: EVP_CIPHER_CTX_new failed\n");
        return -1;
    }

    const EVP_CIPHER *cipher = NULL;

    if (strcmp(algorithm, AES) == 0) {
        cipher = EVP_aes_128_cbc();
    } else if (strcmp(algorithm, CAMELLIA) == 0) {
        cipher = EVP_camellia_128_cbc();
    } else if (strcmp(algorithm, SM4) == 0) {
        cipher = EVP_sm4_cbc();
    } else {
        fprintf(stderr, "ERROR: unknown algorithm: %s\n", algorithm);
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    if (1 != EVP_DecryptInit_ex(ctx, cipher, NULL, key, iv)) {
        fprintf(stderr, "ERROR: EVP_DecryptInit_ex failed\n");
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    int block_size = EVP_CIPHER_block_size(cipher);
    unsigned char *out = malloc((size_t)cd->ciphertext_len + block_size);

```

```

    if (!out) {
        perror("malloc");
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    int outlen = 0, tmplen = 0;
    if (cd->ciphertext_len > 0) {
        if (1 != EVP_DecryptUpdate(ctx, out, &outlen, (unsigned char*)cd-
>ciphertext, cd->ciphertext_len)) {
            fprintf(stderr, "ERROR: EVP_DecryptUpdate failed\n");
            free(out);
            EVP_CIPHER_CTX_free(ctx);
            return -1;
        }
    }

    if (1 != EVP_DecryptFinal_ex(ctx, out + outlen, &tmplen)) {
        // bad padding
        fprintf(stderr, "ERROR: EVP_DecryptFinal_ex failed (bad
padding?)\n");
        free(out);
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }
    outlen += tmplen;

    EVP_CIPHER_CTX_free(ctx);

    free(out);

    double elapsed = get_real_time_msec() - start;
    return elapsed;
}

void testing_aes(int number_file, cipher_data* cd){
    double result;
    int fd;
    char* shared_memory_pointer;
    unsigned char* buffer;
    //opening file
    switch (number_file){
    case 1:
        // open the file
        fd = open(FILE_PATH_1, O_RDONLY);
        if (fd == -1) {

```

```

        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_aes\n");
        exit(EXIT_FAILURE);
        break;
    }

    //mmap
    shared_memory_pointer =
mmap(NULL, FILE_SIZE_1, PROT_READ, MAP_PRIVATE, fd, 0);
    if (shared_memory_pointer == MAP_FAILED) {
        perror("ERROR: Something went wring in mmap in testing_aes\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_1);
    if (!buffer) { perror("malloc"); exit(1); }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_1);
    munmap(shared_memory_pointer, FILE_SIZE_1);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_1;

    //testing
    result = testing_encryption(AES, cd);
    printf("Encryption - AES - %d - %lf \n", FILE_SIZE_1, result);
    result = testing_decryption(AES, cd);
    printf("Decryption - AES - %d - %lf \n", FILE_SIZE_1, result);

    //cleaning memory
    myfree(cd, fd);
    break;

case 2:
    // open the file
    fd = open(FILE_PATH_2, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_aes\n");
        exit(EXIT_FAILURE);
        break;
    }

    //mmap
    shared_memory_pointer =
mmap(NULL, FILE_SIZE_2, PROT_READ, MAP_PRIVATE, fd, 0);
    if (shared_memory_pointer == MAP_FAILED) {

```

```

        perror("ERROR: Something went wring in mmap in testing_aes\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_2);
    if (!buffer) { perror("malloc"); exit(1); }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_2);
    munmap(shared_memory_pointer, FILE_SIZE_2);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_2;

    //testing
    result = testing_encryption(AES,cd);
    printf("Encryption - AES - %d - %lf \n", FILE_SIZE_2, result);
    result = testing_decryption(AES, cd);
    printf("Decryption - AES - %d - %lf \n", FILE_SIZE_2, result);

    //cleaning memory
    myfree(cd,fd);
    break;

case 3:
    // open the file
    fd = open(FILE_PATH_3, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_aes\n");
        exit(EXIT_FAILURE);
    }

    //mmap
    shared_memory_pointer =
mmap(NULL,FILE_SIZE_3,PROT_READ,MAP_PRIVATE,fd,0);
    if (shared_memory_pointer== MAP_FAILED){
        perror("ERROR: Something went wring in mmap in testing_aes\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_3);
    if (!buffer) { perror("malloc"); exit(1); }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_3);
    munmap(shared_memory_pointer, FILE_SIZE_3);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_3;

```

```

//testing
result = testing_encryption(AES, cd);
printf("Encryption - AES - %d - %lf \n", FILE_SIZE_3, result);
result = testing_decryption(AES, cd);
printf("Decryption - AES - %d - %lf \n", FILE_SIZE_3, result);

//cleaning memory
myfree(cd, fd);

break;

default:
    fprintf(stderr, "ERROR: i should not be here :(\n");
    exit(EXIT_FAILURE);
    break;
}
}

void testing_camellia(int number_file, cipher_data* cd){
    double result;
    int fd;
    char* shared_memory_pointer;
    unsigned char* buffer;
    //opening file
    switch (number_file){
    case 1:
        // open the file
        fd = open(FILE_PATH_1, O_RDONLY);
        if (fd == -1) {
            fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_camellia\n");
            exit(EXIT_FAILURE);
            break;
        }

        //mmap
        shared_memory_pointer =
mmap(NULL, FILE_SIZE_1, PROT_READ, MAP_PRIVATE, fd, 0);
        if (shared_memory_pointer == MAP_FAILED){
            perror("ERROR: Something went wring in mmap in
testing_camellia\n");
            close(fd);
            exit(EXIT_FAILURE);
        }
        buffer = (unsigned char*)malloc(FILE_SIZE_1);
        if (!buffer) { perror("malloc"); exit(1); }
    }
}

```

```

memcpy(buffer, shared_memory_pointer, FILE_SIZE_1);
munmap(shared_memory_pointer, FILE_SIZE_1);

cd->plaintext=buffer;
cd->plaintext_len = FILE_SIZE_1;

//testing
result = testing_encryption(CAMELLIA, cd);
printf("Encryption - CAMELLIA - %d - %lf \n", FILE_SIZE_1, result);
result = testing_decryption(CAMELLIA, cd);
printf("Decryption - CAMELLIA - %d - %lf \n", FILE_SIZE_1, result);

//cleaning memory
myfree(cd,fd);

break;

case 2:
    // open the file
    fd = open(FILE_PATH_2, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_camellia\n");
        exit(EXIT_FAILURE);
        break;
    }

    //mmap
    shared_memory_pointer =
mmap(NULL, FILE_SIZE_2, PROT_READ, MAP_PRIVATE, fd, 0);
    if (shared_memory_pointer == MAP_FAILED){
        perror("ERROR: Something went wring in mmap in
testing_camellia\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_2);
    if (!buffer) { perror("malloc"); exit(1); }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_2);
    munmap(shared_memory_pointer, FILE_SIZE_2);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_2;

    //testing
    result = testing_encryption(CAMELLIA, cd);

```

```

printf("Encryption - CAMELLIA - %d - %lf \n", FILE_SIZE_2, result);
result = testing_decryption(CAMELLIA, cd);
printf("Decryption - CAMELLIA - %d - %lf \n", FILE_SIZE_2, result);

//cleaning memory
myfree(cd,fd);

break;

case 3:
    // open the file
    fd = open(FILE_PATH_3, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_camellia\n");
        exit(EXIT_FAILURE);
    }

    //mmap
    shared_memory_pointer =
mmap(NULL, FILE_SIZE_3, PROT_READ, MAP_PRIVATE, fd, 0);
    if (shared_memory_pointer == MAP_FAILED){
        perror("ERROR: Something went wrong in mmap in
testing_camellia\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_3);
    if (!buffer) {
        fprintf(stderr, "ERROR: Something went wring in mmap in
testing_camellia\n");
        exit(EXIT_FAILURE);
    }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_3);
    munmap(shared_memory_pointer, FILE_SIZE_3);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_3;

    //testing
    result = testing_encryption(CAMELLIA, cd);
    printf("Encryption - CAMELLIA - %d - %lf \n", FILE_SIZE_3, result);
    result = testing_decryption(CAMELLIA, cd);
    printf("Decryption - CAMELLIA - %d - %lf \n", FILE_SIZE_3, result);

    //cleaning memory

```

```

        myfree(cd, fd);

        break;

    default:
        fprintf(stderr, "ERROR: i should not be here :(\n");
        exit(EXIT_FAILURE);
        break;
    }
}

void testing_sm4(int number_file, cipher_data* cd){
    double result;
    int fd;
    char* shared_memory_pointer;
    unsigned char* buffer;
    //opening file
    switch (number_file){
    case 1:
        // open the file
        fd = open(FILE_PATH_1, O_RDONLY);
        if (fd == -1) {
            fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_sm4\n");
            exit(EXIT_FAILURE);
            break;
        }

        //mmap
        shared_memory_pointer =
mmap(NULL, FILE_SIZE_1, PROT_READ, MAP_PRIVATE, fd, 0);
        if (shared_memory_pointer == MAP_FAILED){
            perror("ERROR: Something went wring in mmap in testing_sm4\n");
            close(fd);
            exit(EXIT_FAILURE);
        }
        buffer = (unsigned char*)malloc(FILE_SIZE_1);
        if (!buffer) { perror("malloc"); exit(1); }
        memcpy(buffer, shared_memory_pointer, FILE_SIZE_1);
        munmap(shared_memory_pointer, FILE_SIZE_1);

        cd->plaintext=buffer;
        cd->plaintext_len = FILE_SIZE_1;

        //testing
        result = testing_encryption(SM4, cd);
        printf("Encryption - SM4 - %d - %lf \n", FILE_SIZE_1, result);
    }
}

```

```

    result = testing_decryption(SM4, cd);
    printf("Decryption - SM4 - %d - %lf \n", FILE_SIZE_1, result);

    //cleaning memory
    myfree(cd,fd);

    break;

case 2:
    // open the file
    fd = open(FILE_PATH_2, O_RDONLY);
    if (fd == -1) {
        fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_sm4\n");
        exit(EXIT_FAILURE);
        break;
    }

    //mmap
    shared_memory_pointer =
mmap(NULL, FILE_SIZE_2, PROT_READ, MAP_PRIVATE, fd, 0);
    if (shared_memory_pointer == MAP_FAILED) {
        perror("ERROR: Something went wring in mmap in testing_sm4\n");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer = (unsigned char*)malloc(FILE_SIZE_2);
    if (!buffer) { perror("malloc"); exit(1); }
    memcpy(buffer, shared_memory_pointer, FILE_SIZE_2);
    munmap(shared_memory_pointer, FILE_SIZE_2);

    cd->plaintext=buffer;
    cd->plaintext_len = FILE_SIZE_2;

    //testing
    result = testing_encryption(SM4,cd);
    printf("Encryption - SM4 - %d - %lf \n", FILE_SIZE_2, result);
    result = testing_decryption(SM4, cd);
    printf("Decryption - SM4 - %d - %lf \n", FILE_SIZE_2, result);

    //cleaning memory
    myfree(cd,fd);

    break;

case 3:

```

```

// open the file
fd = open(FILE_PATH_3, O_RDONLY);
if (fd == -1) {
    fprintf(stderr, "ERROR: Something went wrong opening in opening
file in testing_sm4\n");
    exit(EXIT_FAILURE);
}

//mmap
shared_memory_pointer =
mmap(NULL, FILE_SIZE_3, PROT_READ, MAP_PRIVATE, fd, 0);
if (shared_memory_pointer == MAP_FAILED){
    perror("ERROR: Something went wring in mmap in testing_sm4\n");
    close(fd);
    exit(EXIT_FAILURE);
}
buffer = (unsigned char*)malloc(FILE_SIZE_3);
if (!buffer) { perror("malloc"); exit(1); }
memcpy(buffer, shared_memory_pointer, FILE_SIZE_3);
munmap(shared_memory_pointer, FILE_SIZE_3);

cd->plaintext=buffer;
cd->plaintext_len = FILE_SIZE_3;

//testing
result = testing_encryption(SM4, cd);
printf("Encryption - SM4 - %d - %lf \n", FILE_SIZE_3, result);
result = testing_decryption(SM4, cd);
printf("Decryption - SM4 - %d - %lf \n", FILE_SIZE_3, result);

//cleaning memory
myfree(cd, fd);
break;

default:
    fprintf(stderr, "ERROR: i should not be here :(\n");
    exit(EXIT_FAILURE);
    break;
}
}

int main(){
    //working with the KEY and the IV
    unsigned char key[KEY_LENGTH];

```

```

    unsigned char iv[IV_LENGTH];

    if (RAND_bytes(key, KEY_LENGTH) !=1){ //randomizing key
        fprintf(stderr, "ERROR: something went wrong with
RAND_bytes(key,...)\n");
        return 1;
    }
    if (RAND_bytes(iv, IV_LENGTH) !=1){ //randomizing IV
        fprintf(stderr, "ERROR: something went wrong with
RAND_bytes(iv,...)\n");
        return 1;
    }

    cipher_data* cd = (cipher_data*)malloc(sizeof(cipher_data));
    cd->key =key;
    cd->iv=iv;
    cd->plaintext = NULL;
    cd->ciphertext = NULL;
    cd->plaintext_len = 0;
    cd->ciphertext_len = 0;

    //starting testing
    printf("-----STARTING TESTING-----\n");
    printf("TYPE - ALGORITHM - FILE_SIZE - RESULT(ms)\n");
    for (int i =1; i<=3; i++){
        for (int j = 1; j<=3; j++){
            if (i==1) testing_aes(j, cd);
            else if (i==2) testing_camellia(j,cd);
            else if (i==3) testing_sm4(j, cd);
            else{
                fprintf(stderr, "ERROR: i should not be here :(\n");
                return 1;
            }
        }
    }
    printf("-----\n");
    return 0;
}

```